MILESTONE D3 · PART 1

Order Service

The Order Service manages one errand for its whole life. It records who asked, who is delivering, where to pick up and drop off, how many credits are offered, and the status. User profiles, store records, and credit balances stay in their own services.

1 Database choice and schema

The database is PostgreSQL. The choice follows the shape of the data, the consistency rules, and the size of the system.

- **Shape.** An errand is one structured row: one requester, at most one courier, one store, one drop-off, one credit amount, and one status. The usual reads are the open board, one student's requests, and one courier's jobs.
- **Consistency.** Two couriers accepting the same errand, or an accept arriving with a cancel or an expiry, must have a single winner. PostgreSQL locks the row and commits the new status with its event. The event cannot be lost if the message broker is down.
- **Scale.** This is a campus board. The difficult requirement is a correct result under concurrent updates, and PostgreSQL covers that volume without a distributed store.

Order row

The **orders** table is the errand. A check constraint limits status to the eight names below, and another requires credit_amount to be greater than zero. The API also caps the amount at 1000.

Column	What it stores
id	Primary key. UUID.
requester_id	User who placed the errand. Always set. Owned by User Service.
courier_id	User who accepted it. Empty until the accept commits.
pickup_supplier_id	Store to collect from. An id from Supplier Service.
delivery_label, delivery_latitude, delivery_longitude, delivery_details	The drop-off, written on the order itself.
item_description	What the courier should collect.
credit_amount	Credits offered for this errand. Integer, greater than 0.
status	PENDING_CREDIT, OPEN, REJECTED, ACCEPTED, PICKED_UP, COMPLETED, CANCELLED, or EXPIRED.
idempotency_key, request_hash	A retried create with the same key and body returns the original order. Unique per requester and key.
correlation_id	Ties this order to the events written for it.
rejection_reason	Why Credit refused the hold, when that happens.
expires_at and the step times	expires_at is one hour after creation. accepted_at, picked_up_at, completed_at, cancelled_at, cancelled_by, and expired_at record each later step.

Two other tables sit in the same database. **outbox_events** stores the event in the same commit as the status change. A worker publishes unpublished rows in id order. **processed_events** stores incoming event ids that have already been applied, so a repeat does nothing.

Indexes follow the reads: requester plus created time, courier plus created time, and a partial index of created time for rows whose status is open.

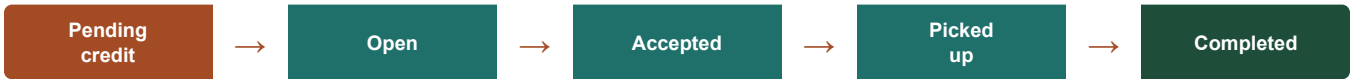
References to other services

The order stores ids, not copies of the other services' records. There is no foreign key across services.

- **Users.** requester_id, courier_id, and cancelled_by are User Service ids. The order does not store a name, email, or phone number. A later profile edit leaves the order as it is. Each request checks the caller again, so a suspended or inactive account cannot act, while orders already created stay in place.
- **Stores.** pickup_supplier_id is checked against Supplier Service only when the order is created. The store must be active then. A later rename, closure, or removal does not rewrite or cancel the errand. The drop-off coordinates belong to the order, not to the store.
- **Credits.** The order stores the offered amount. Balances and reservations live in Credit Service. The link between them is the order id carried on each event.

2 State transitions

Every transition updates the order and writes its event in one database transaction. The API process does not talk to RabbitMQ. Invalid transitions leave the row unchanged and are logged.



Rejected, cancelled, and expired stop there. Nothing leaves those states, and a completed order does not change again.

New → Pending credit

Condition. The caller is an active user, the store is active, the drop-off is inside the campus box, the credit amount is from 1 to 1000, and the idempotency key is new or repeats the same body.

Credit. The amount is stored and order.created is written, including expires_at. That event asks Credit to reserve the amount. This service does not change a balance. The order stays pending until Credit answers.

Who. The requester.

Rejected. Bad input or an inactive store is 422. A bad session is 401. The same key with a different body is 409. If User Service, Supplier Service, or the database cannot be reached, the response is 503.

Pending credit → Open

Condition. A credit.reserved event arrives and the order is still pending.

Credit. The hold stands. order.opened is written. The balance is still not stored here.

Who. The worker, applying Credit's event.

Rejected. If the order has already moved on, the event is recorded and the status stays as it is.

Pending credit → Rejected

Condition. A credit.reservation_failed event arrives and the order is still pending.

Credit. The order does not open. The reason is stored and order.rejected is written.

Who. The worker, applying Credit's event.

Rejected. If the order is no longer pending, the status does not change.

Open → Accepted

Condition. The status is open, and the caller is not the requester.

Credit. No credit movement in this service. order.accepted records the courier.

Who. Any other active user.

Rejected. The requester gets 403. If another courier already accepted, the response is 409. Any other status is 409 for someone who can see the order, and 404 for everyone else. The same courier repeating the accept gets the current order and no second event.

Accepted → Picked up

Condition. The caller is the assigned courier, and the status is accepted.

Credit. No credit movement. order.picked_up is written.

Who. The assigned courier.

Rejected. Someone who can see the order but is not the courier gets 403. Anyone else gets 404. The wrong status is 409. A repeat by the same courier returns the order.

Picked up → Completed

Condition. The caller is the requester, and the status is picked up.

Credit. This service does not transfer credits. order.completed tells Credit to pay the reserved amount to the courier.

Who. The requester.

Rejected. Anyone else gets 403 or 404. The wrong status is 409. A second completion by the requester returns the same order and does not write another event.

Pending credit, open, or accepted → Cancelled

Condition. The caller is the requester and the status is one of those three.

Credit. order.cancelled tells Credit to release the reservation. The event includes the previous status and who cancelled.

Who. The requester.

Rejected. After pickup, the requester gets 403. A terminal order is 409. Repeating the cancel returns the same order.

Any non-terminal state → Cancelled

Condition. The caller is an admin, and the order is not completed, cancelled, expired, or rejected. This includes an order that has already been picked up.

Credit. The same cancel event is written. If the previous status was picked up, that is the signal for Credit to compensate the courier as well as release the hold.

Who. An admin.

Rejected. A terminal order is 409.

Pending credit or open → Expired

Condition. One hour has passed since creation, and the status is still pending or open.

Credit. order.expired tells Credit to release the reservation.

Who. The worker. The original order.created message waits in a one-hour delay queue, then comes back.

Rejected. If the order was accepted, cancelled, or otherwise moved on, the message changes nothing. The event id is stored so a redelivery does not try again.

Changing the drop-off is not a status change. The requester may replace it while the order is pending, open, or accepted. The new pin must still be inside the campus box. No event is written.

3 API design, queries, and access control

Every route except /health and /ready requires Authorization: Bearer. The token is forwarded to User Service GET /users/me. The order body never supplies the caller's id. Errors are returned as {"detail": "..."}.

Method	Path	Who	Result
POST	/orders	Requester	201. A retry with the same key and body returns 200 and the original order.
GET	/orders/open	Any active user	Open errands except the caller's, newest first. Pending orders stay hidden.
GET	/orders?role=requester	Caller	Errands that caller requested.

Method	Path	Who	Result
GET	/orders?role=courier	Caller	Errands assigned to that caller.
GET	/orders/{id}	Requester, assigned courier, or admin	That order. Anyone else gets 404, even when the id exists.
PATCH	/orders/{id}/delivery	Requester, before pickup	Replaces the drop-off. No event.
POST	/orders/{id}/accept	Anyone except the requester	The first committed accept wins. The others get 409.
POST	/orders/{id}/pickup	Assigned courier	Marks the order picked up.
POST	/orders/{id}/complete	Requester	Marks it completed. Does not move balances.
POST	/orders/{id}/cancel	Requester before pickup, or an admin until it is closed	Cancels the order.

Lists are pages of {items, total, limit, offset}. Limit is from 1 to 100. POST /orders requires Idempotency-Key, at most 128 characters. X-Correlation-ID is optional and must be a UUID.

Validation and conflicts

- Unknown JSON fields are rejected. Text is stripped and must not be blank. credit_amount is an integer from 1 to 1000.
- The drop-off must sit inside latitude 1.288–1.310 and longitude 103.750–103.788, the box that covers the seeded stores and University Town.
- The pickup id must be an active supplier. A missing supplier is 422. Any other failure from Supplier Service is 503.
- 401 means the session is missing, invalid, or the account is not active. 403 means the caller is real but the action is not theirs. 404 hides an order the caller is not a party to. 409 is an illegal status change, or an idempotency key reused with a different body. 422 is bad input. 503 means User Service, Supplier Service, or the database could not answer.

Identity and the caller’s link to the order

Authorization uses both facts. The session says who the caller is and whether they are a user or an admin. The order row says whether they are its requester or its courier. Both checks run in the service on every call.

- A requester cannot accept their own order. The open list also leaves their own errands out, so the board does not offer them. The accept route still returns 403 if they call it.
- Only the assigned courier can pick up. Only the requester can complete, change the drop-off, or cancel before pickup.
- An unrelated student who guesses an id gets 404, the same response as a missing order.
- An admin can open any order by id and can cancel it until it is closed, including after pickup. List routes still return only the caller’s own orders. There is no route that lists another student’s orders.

The same student requesting and delivering

One account has one id. That student may create errands and, at the same time, accept other people’s open errands. GET /orders?role=requester and GET /orders?role=courier are two lists. Accepting is refused only when the caller’s id is the requester id on that order. Doing both jobs is normal. Doing both jobs on the same order is not.

4 Concurrency and corner cases

Only one courier can accept an order because the accept locks the row. The request runs SELECT … FOR UPDATE, checks the status, then commits the new status and order.accepted together. PostgreSQL holds any second accept until the first transaction finishes. That second request then sees the status is accepted and the courier is someone else, and it returns 409. It does not overwrite the winner.

The loser is told they lost. A 404 is reserved for an order they are not allowed to see. Two couriers who both saw the errand while it was open therefore get a conflict, not a missing order.

Accept at the same moment as cancel or expiry

Cancel takes the same row lock in the API transaction. Expiry takes it in the worker. One commit wins.

- If cancel or expiry commits first, the accept no longer sees an open order. A courier who is not a party to it receives 404. No accept event is written.
- If accept commits first, the requester can still cancel, because an accepted order is cancellable by its requester. Expiry cannot undo the accept: expiry changes only pending and open orders. The delayed message is recorded and the order stays accepted.
- The same rule covers credit.reserved arriving after expiry. Opening happens only from pending credit, so an expired order is not reopened.

A completion request submitted twice

The second request locks the row, sees that this requester already completed it, and returns that order with 200. It does not write a second order.completed, so Credit is not asked to pay twice. A repeated pickup or a repeated cancel by the same person returns the current order in the same way.

Two identical creates are covered by the unique key on requester and idempotency key. One insert commits. The other hits the constraint, loads the saved row, and returns it when the body hash matches.

5 Integration and demonstration

Order Service calls User Service and Supplier Service while it handles the request. It does not call Credit Service over HTTP. Credit is told, and answers, through events on the foc.events topic exchange. The worker publishes those events from the outbox and applies credit.reserved, credit.reservation_failed, and the delayed order.created message.

Partner	When Order Service uses it	What changes
User Service	Every authenticated call. GET /users/me with the bearer token.	The returned id and role become the caller. A user id in the body is ignored.
Supplier Service	Create only. GET /suppliers/{id}.	An active store is required. The order keeps the store id. Later store edits do not flow back into the order.
Credit Service	After the commit, through events. Not during the HTTP call.	order.created asks for a reserve. order.completed asks for payment to the courier. order.cancelled and order.expired ask for a release. A cancel whose previous status is picked up also asks Credit to compensate the courier.

What a demo can show

- Create with a bearer token, an Idempotency-Key, an active store, a campus drop-off, and a credit amount. The response is 201 and the status is pending credit. The same commit contains an order.created row in outbox_events. Repeating the same key and body returns 200 and does not create a second order.
- When Credit publishes credit.reserved, the worker moves the order to open and writes order.opened. It then appears on GET /orders/open for other students, and not for the requester. credit.reservation_failed moves it to rejected instead.
- A second student calls POST /orders/{id}/accept and becomes the courier. The requester calling accept gets 403. Another student calling accept gets 409. Pickup by the courier, then complete by the requester, writes order.picked_up and order.completed. The order row shows the new status and the step times. The credit amount on the order does not change.
- Cancel before pickup, or an admin cancel, writes order.cancelled with the previous status. An order still pending or open one hour after creation becomes expired. Those events are what Credit uses to release the hold, and order.completed is what it uses to pay the courier.

Credit balances change in Credit Service when it consumes those events. Order Service can show the order, the status, and the outbox event on its own. The matching balance change is visible once Credit Service is running beside it.